

Troubleshooting OTel Pipelines

Series: OTEL | Notebook: 8 of 8 | Created: January 2026

Debugging OpenTelemetry Data Flows

When telemetry data doesn't arrive as expected, systematic troubleshooting is essential. This notebook covers common issues, diagnostic techniques, and resolution steps for OpenTelemetry pipelines.

Table of Contents

- 1. Troubleshooting Methodology
- 2. Collector Issues
- 3. SDK Issues
- 4. Connectivity Issues
- 5. Data Quality Issues
- 6. Performance Issues
- 7. Diagnostic Tools
- 8. Common Fixes

Prerequisites

Requirement	Details
Knowledge	OTEL-01 through OTEL-07
Access	Collector logs, application logs

1. Troubleshooting Methodology

Systematic Approach

- ```
1. Identify the symptom
 ↓
2. Check component by component
 SDK → Collector → Backend
 ↓
3. Verify configuration
 ↓
4. Test connectivity
 ↓
5. Apply fix
 ↓
6. Verify resolution
```

## Symptom Categories

| Symptom           | Likely Issue        | Start Here |
|-------------------|---------------------|------------|
| No data at all    | Connectivity, auth  | Section 4  |
| Partial data      | Filtering, sampling | Section 5  |
| Delayed data      | Performance         | Section 6  |
| Wrong data        | Configuration       | Section 5  |
| Collector crashes | Resources           | Section 6  |

## 2. Collector Issues

---

### Collector Won't Start

Check configuration syntax:

```
Validate config
otelcol-contrib validate --config=config.yaml
```

### Common configuration errors:

| Error                             | Cause                   | Fix                                          |
|-----------------------------------|-------------------------|----------------------------------------------|
| <code>unknown type</code>         | Invalid component name  | Check spelling, verify contrib has component |
| <code>duplicate key</code>        | YAML syntax error       | Check indentation                            |
| <code>no pipelines defined</code> | Missing service section | Add service.pipelines                        |

### Enable Debug Logging

```
service:
 telemetry:
 logs:
 level: debug
```

### Collector Dropping Data

#### Check queue status:

```
If using zpages extension
curl http://localhost:55679/debug/tracez
```

#### Common causes:

| Cause           | Indicator           | Fix                              |
|-----------------|---------------------|----------------------------------|
| Memory pressure | OOM, restarts       | Increase memory_limiter limits   |
| Export failure  | Retry logs          | Check backend connectivity       |
| Queue full      | Queue overflow logs | Increase queue size or consumers |

## 3. SDK Issues

---

### No Spans Generated

#### Python:

```
Enable SDK debug logging
import logging
logging.basicConfig(level=logging.DEBUG)

Verify tracer provider is set
from opentelemetry import trace
print(trace.get_tracer_provider())
Should NOT be NoOpTracerProvider
```

## Java:

```
Enable agent debug
java -javaagent:opentelemetry-javaagent.jar \
 -Dotel.javaagent.debug=true \
 -jar app.jar
```

## Common SDK Issues

| Issue         | Cause               | Fix                                     |
|---------------|---------------------|-----------------------------------------|
| NoOpTracer    | Provider not set    | Call <code>set_tracer_provider()</code> |
| No export     | Processor not added | Add <code>BatchSpanProcessor</code>     |
| Missing spans | Not in context      | Use <code>start_as_current_span</code>  |
| Lost context  | Async not handled   | Use context propagation                 |

## Async Context Loss

```
Wrong – loses context
async def bad_example():
 with tracer.start_as_current_span("parent"):
 await asyncio.gather(
 child_task(), # No context
 child_task()
)

Right – preserves context
async def good_example():
 with tracer.start_as_current_span("parent"):
 token = context.attach(context.get_current())
 try:
 await asyncio.gather(
 child_task(),
 child_task()
)
 finally:
 context.detach(token)
```

## 4. Connectivity Issues

---

### Test Collector Connectivity

```
Test OTLP endpoint
curl -v http://collector:4318/v1/traces \
 -X POST \
 -H "Content-Type: application/json" \
 -d '{}'
```

```
Test gRPC (with grpcurl)
grpcurl -plaintext collector:4317 list
```

## Test Dynatrace Connectivity

```
Test OTLP endpoint
curl -v https://abc12345.live.dynatrace.com/api/v2/otlp/v1/traces \
 -X POST \
 -H "Authorization: Api-Token dt0c01.XXX..." \
 -H "Content-Type: application/json" \
 -d '{}'
```

# Expected: 200 OK (empty body is fine)

## Common Connectivity Errors

| Error              | Cause            | Fix                                  |
|--------------------|------------------|--------------------------------------|
| connection refused | Wrong port/host  | Verify endpoint                      |
| 401 Unauthorized   | Invalid token    | Check token, regenerate              |
| 403 Forbidden      | Missing scope    | Add required scope to token          |
| certificate error  | TLS issue        | Check certs, disable verify for test |
| timeout            | Network/firewall | Check firewall rules                 |

## Proxy Issues

```
Collector proxy configuration
exporters:
 otlphttp:
 endpoint: https://tenant.live.dynatrace.com/api/v2/otlp
 proxy_url: http://proxy.example.com:8080
```

```
Or via environment
export HTTPS_PROXY=http://proxy.example.com:8080
```

## 5. Data Quality Issues

---

### Missing Attributes

Debug exporter to see data:

```
exporters:
 debug:
 verbosity: detailed

service:
 pipelines:
 traces:
 exporters: [debug, otlphttp]
```

### service.name Missing

Without `service.name`, Dynatrace can't create service entities:

```
Add default service.name in collector
processors:
 resource:
 attributes:
 - key: service.name
 value: "unknown-service"
 action: insert # Only adds if missing
```

### Wrong Time Zone

```
Ensure collector uses UTC
In Kubernetes:
env:
 - name: TZ
 value: UTC
```

### Data Being Filtered

Check filter processor configuration:

```
processors:
 filter:
 traces:
 span:
 # This filters OUT matching spans
 - 'attributes["http.url"] == "/health"'
```

## 6. Performance Issues

---

### Collector Memory Issues

#### Symptoms:

- OOMKilled restarts
- Slow processing
- Data drops

#### Fixes:

```
processors:
 memory_limiter:
 check_interval: 1s
 limit_mib: 1500 # Increase limit
 spike_limit_mib: 500 # Allow spikes

 batch:
 timeout: 5s # Faster flush
 send_batch_size: 500 # Smaller batches
```

### High Latency

#### Causes and fixes:

| Cause               | Indicator       | Fix                           |
|---------------------|-----------------|-------------------------------|
| Large batches       | Delayed export  | Reduce batch size             |
| Slow export         | Backend latency | Increase parallelism          |
| Too many processors | Processing time | Remove unnecessary processors |



```
exporters:
 otlphttp:
 sending_queue:
 num_consumers: 10 # More parallel exports
```

## SDK Overhead

```
Use sampling to reduce volume
from opentelemetry.sdk.trace.sampling import TraceIdRatioBased

provider = TracerProvider(
 sampler=TraceIdRatioBased(0.1), # 10% sampling
 resource=resource
)
```

## 7. Diagnostic Tools

---

### Collector zpages

```
extensions:
 zpages:
 endpoint: 0.0.0.0:55679

service:
 extensions: [zpages]
```

Access:

- <http://collector:55679/debug/servicez> - Service info
- <http://collector:55679/debug/pipelinez> - Pipeline status
- <http://collector:55679/debug/extensionz> - Extensions
- <http://collector:55679/debug/tracez> - Trace samples

### Health Check

```
extensions:
 health_check:
 endpoint: 0.0.0.0:13133
```

```
curl http://collector:13133/
{"status":"Server available"}
```

## pprof Profiling

```
extensions:
 pprof:
 endpoint: 0.0.0.0:1777
```

```
go tool pprof http://collector:1777/debug/pprof/heap
```

```
// Check OTel data arrival
fetch spans, from: now() - 1h
| filter isNotNull(otel.library.name)
| summarize count(), by:{bin(timestamp, 5m)}
| sort timestamp asc
```

```
// Find services without service.name
fetch spans, from: now() - 1h
| filter isNull(service.name) or service.name == ""
| summarize count(), by:{otel.library.name}
| sort count() desc
```

## 8. Common Fixes

---

### Quick Reference

| Problem           | Fix                                        |
|-------------------|--------------------------------------------|
| No data at all    | Check endpoint, auth token, network        |
| No service entity | Add service.name resource attribute        |
| Collector OOM     | Increase memory_limiter, reduce batch size |
| High cardinality  | Remove dynamic attributes                  |
| Context lost      | Use context propagation in async           |
| Wrong time        | Set TZ=UTC                                 |
| SSL errors        | Check certs, add CA                        |

### Collector Config Template

A production-ready template:

```
receivers:
 otlp:
 protocols:
 grpc:
 endpoint: 0.0.0.0:4317
 http:
 endpoint: 0.0.0.0:4318

processors:
 memory_limiter:
 check_interval: 1s
 limit_mib: 1500
 spike_limit_mib: 500
 batch:
 timeout: 10s
 send_batch_size: 1000
 resource:
 attributes:
 - key: service.name
 value: unknown
 action: insert

exporters:
 otlphttp:
 endpoint: https://${DT_ENV}.live.dynatrace.com/api/v2/otlp
 headers:
 Authorization: Api-Token ${DT_TOKEN}
 retry_on_failure:
 enabled: true
 initial_interval: 5s
 max_interval: 30s

extensions:
 health_check:
 zpages:

service:
 extensions: [health_check, zpages]
 telemetry:
 logs:
 level: info
 pipelines:
 traces:
 receivers: [otlp]
 processors: [memory_limiter, resource, batch]
 exporters: [otlphttp]
 metrics:
```

```
receivers: [otlp]
processors: [memory_limiter, resource, batch]
exporters: [otlphttp]
logs:
 receivers: [otlp]
 processors: [memory_limiter, resource, batch]
 exporters: [otlphttp]
```

---

## Summary

---

In this notebook, you learned:

- Systematic troubleshooting methodology
- Collector debugging and configuration validation
- SDK issue diagnosis and async context handling
- Connectivity testing and common errors
- Data quality issue resolution
- Performance tuning for memory and latency
- Diagnostic tools: zpages, health check, pprof
- Quick fixes for common problems

---

## References

---

- [Collector Troubleshooting](#)
- [SDK Debugging](#)
- [Dynatrace OTel Troubleshooting](#)

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*